

Propuestas Laboratorio 3: Lógica Secuencial y Controladores

Estudiantes: Ana Melissa Vásquez Rojas, Dario Garro Moya

Para el problema se diseñan dos módulos: un controlador de VGA, para una resolución de 640x480 pixeles, así como un controlador (máquina de estados finita) utilizando el controlador de VGA.

Controlador VGA

El formato VGA despliega las imágenes mediante un barrido secuencial de píxeles en la pantalla. Este barrido inicia en la esquina superior izquierda y se realiza línea por línea hasta llegar a la parte inferior de la pantalla.

Cada línea está compuesta por 640 píxeles visibles, que se generan uno a uno de izquierda a derecha. Una vez completada la línea, se agrega un tiempo de píxeles no visibles (front porch, back porch y pulso de sincronía horizontal), en lo que se pasa a la siguiente línea.

Este proceso se repite hasta completar las 480 líneas visibles de la pantalla. De igual manera, al final de cada línea se añaden líneas no visibles que forman parte del intervalo vertical (porches y pulso de sincronía vertical). Cuando se recorren las 525 líneas totales (480 visibles + 45 de control), se completa un cuadro y el barrido regresa a la parte superior, comenzando nuevamente el proceso.

Este mecanismo de barrido puede representarse gráficamente mediante un diagrama de pantalla, en el cual se distinguen las zonas visibles y las secciones no visibles correspondientes a los porches y a las señales de sincronización.

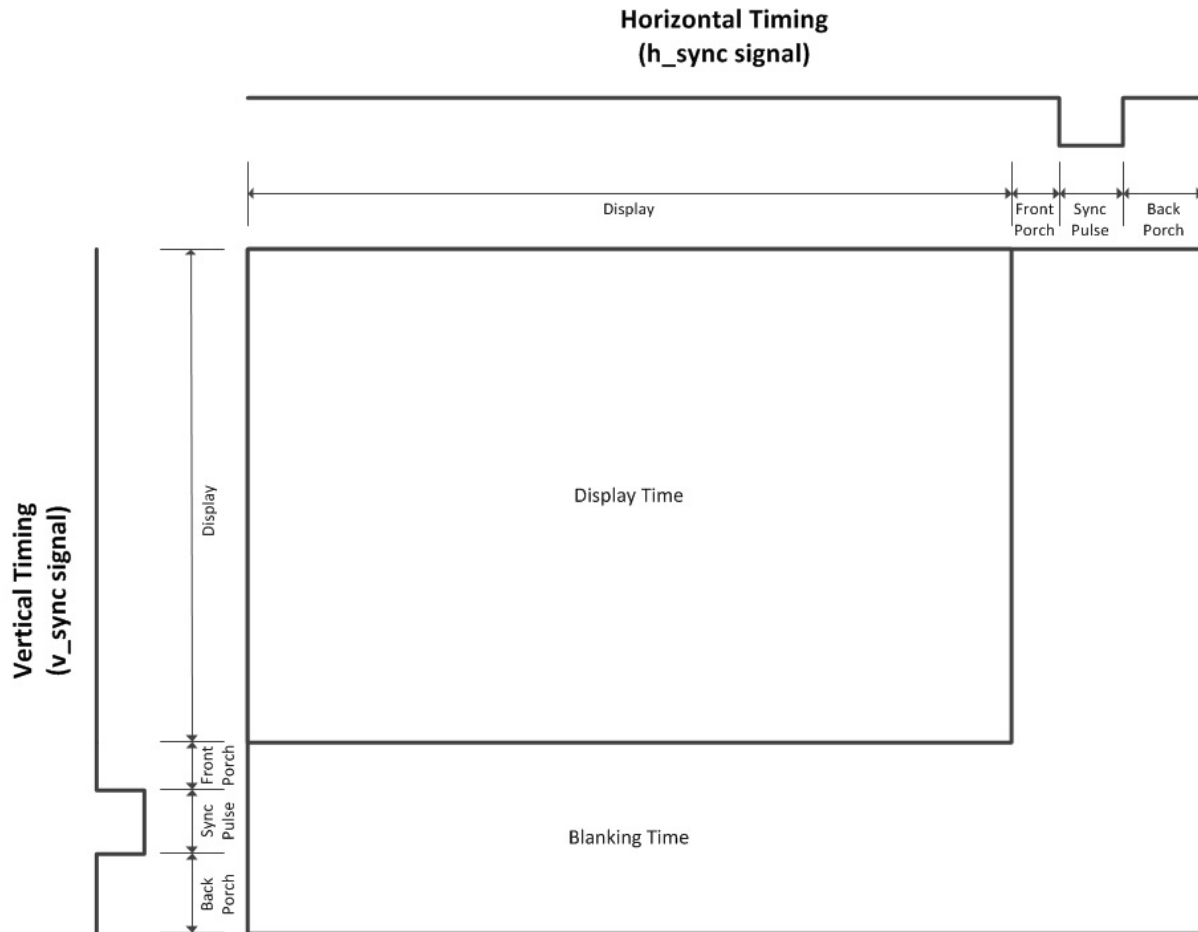


Figura 1. Diagrama de pantalla.

Si asumimos una resolución VGA de 640 x 480 píxeles a 60 Hz, se tienen los siguientes valores:

Pixel clock	Anchura (pixels)				Altura (líneas)			
	Visibles	No visibles			Visibles	No visibles		
		Front Porch	h_sync	Back Porch		Front Porch	v_sync	Back Porch
25.175 MHz	640	16	96	48	480	10	2	33

Por lo que la frecuencia es:

$$\frac{1}{\frac{640+16+96+48}{25175000} \times (480 + 10 + 2 + 33)} = 59.940 \text{ Hz} \approx 60 \text{ Hz}$$

Para coordinar este proceso se emplean dos señales de sincronización, la sincronización horizontal (h_sync), que recorre cada posición, y la sincronización vertical (v_sync), que indica el inicio de una nueva fila.

A continuación, se presentan los diagramas de tiempo correspondientes a estas señales de sincronización.

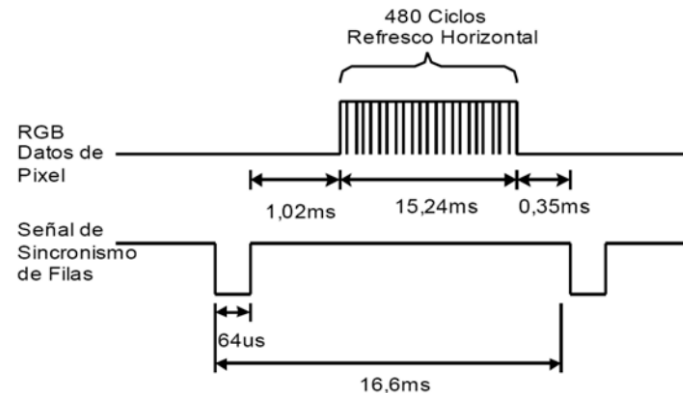


Figura 2. Diagrama de tiempo para la señal v_sync

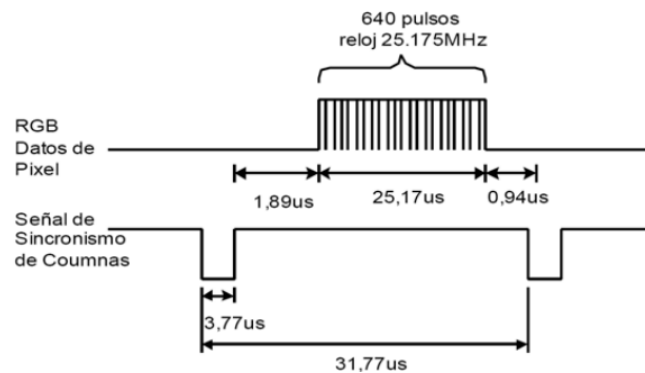


Figura 3. Diagrama de tiempo de la señal h_sync

Una vez descrito el funcionamiento interno del controlador VGA y los elementos que lo conforman, es necesario mostrar cómo este módulo se comunica físicamente la FPGA.

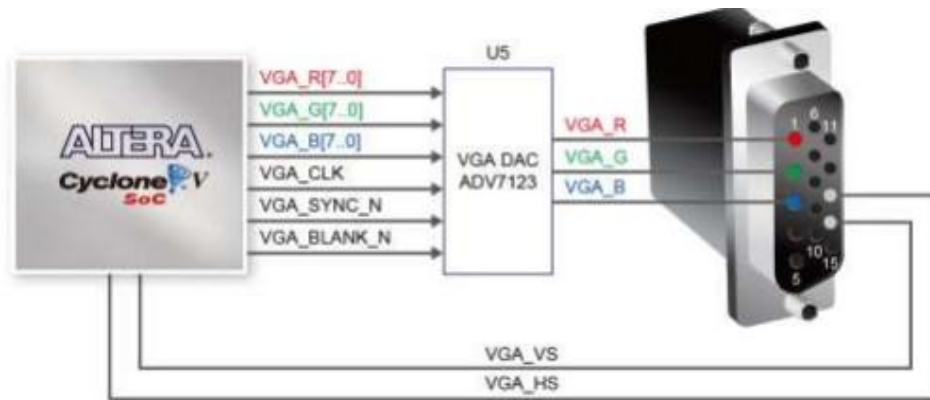


Figura 5. Conexiones entre la FPGA y VGA

A partir de lo anterior, se propone el siguiente diseño para el controlador VGA:

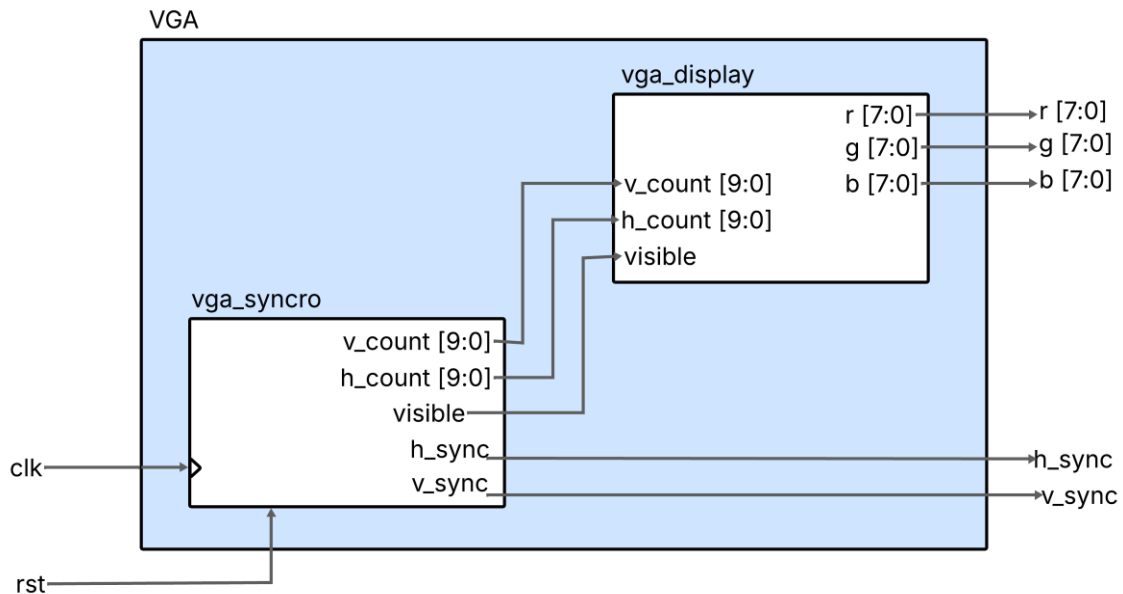


Figura 4. Diagrama diseño controlador VGA.

Para realizar el vga_syncro se tendría lo siguiente:

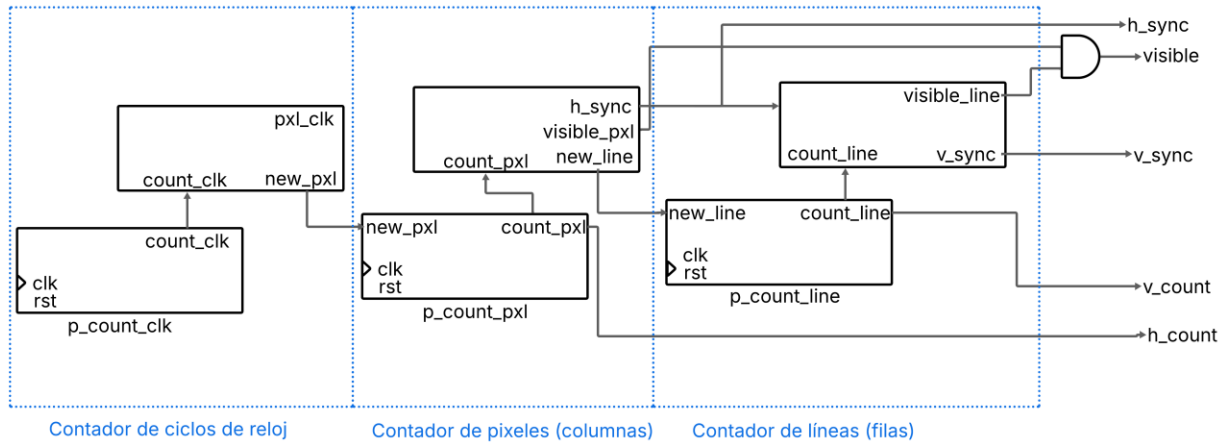


Figura 5. Diagrama de bloques para vga_syncro.

En el contador de ciclos de reloj (count_clk), se genera la señal que indica la llegada de un nuevo píxel. Esta señal (new_pxl) va al contador de píxeles (count_pxl), encargado de recorrer los 800 píxeles de cada línea. A su vez, la señal visible_pxl determina si la posición actual está dentro de la zona visible de la pantalla.

Cuando el contador de píxeles completa una línea, se activa la señal new_line, que habilita al contador de líneas (count_line). Este módulo cuenta hasta 525 líneas en total, de las cuales 480 corresponden a la zona visible. La señal visible_line indica si la línea actual se encuentra dentro de esa región activa.

Finalmente, a partir de las señales de control generadas por los contadores y los comparadores, se producen las señales de sincronización horizontal (h_sync) y vertical (v_sync), así como las coordenadas actuales de columna (h_count) y fila (v_count), junto con la señal visible que habilita la salida de color únicamente en la ventana activa.

Propuesta 1

Se diseñó un controlador principal basado en una máquina de estados finita (FSM) maneja la lógica del juego de cartas, mientras se comunica con el controlador VGA para desplegar la información en pantalla.

Controlador principal

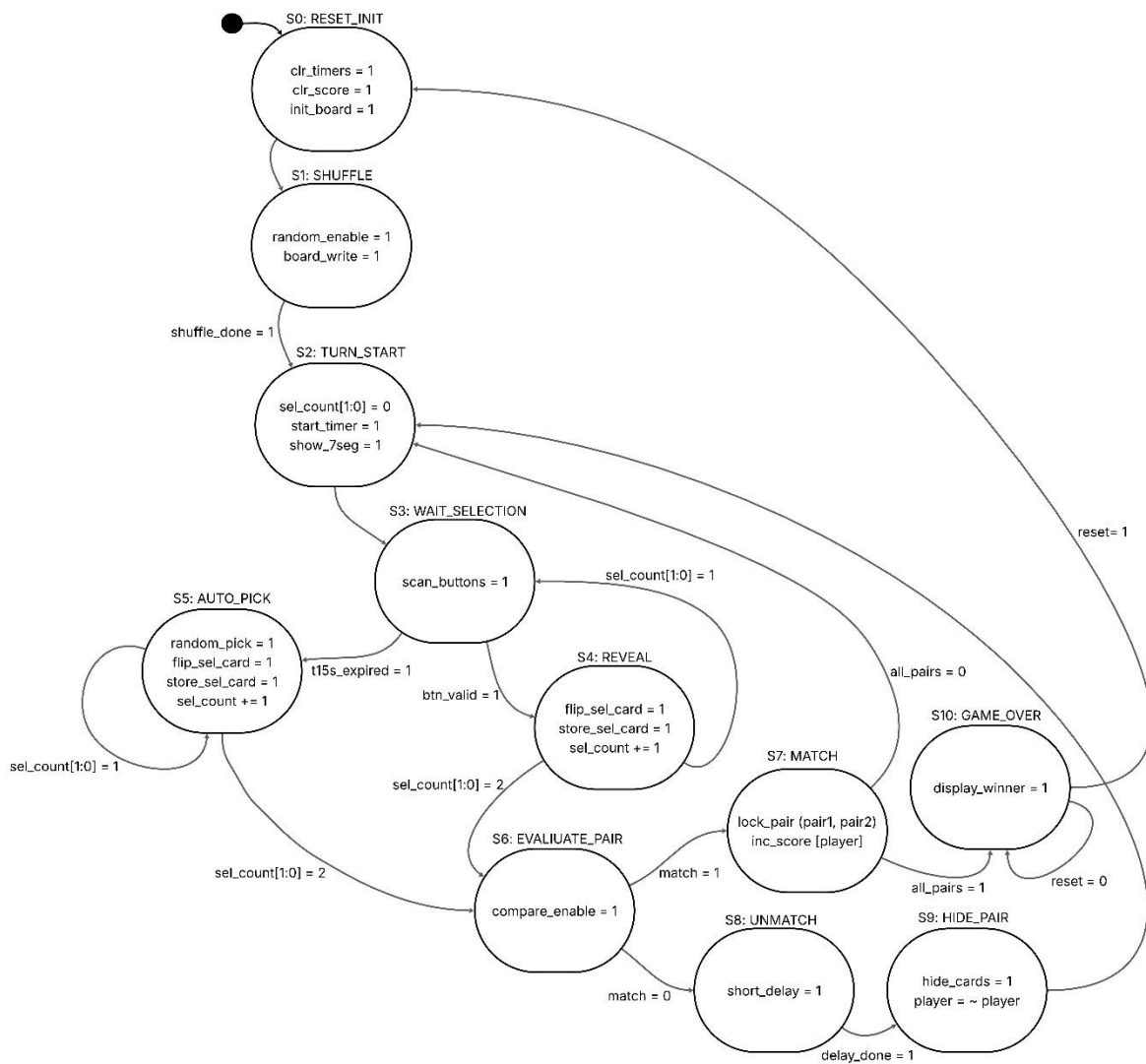


Figura 6. Máquina de Estado Finito para la propuesta 1.

A continuación, se muestra una tabla con la codificación de cada uno de los estados:

Estado	Nombre	Binario (4bits)
S0	RESET_INIT	0000
S1	SHUFFLE	0001
S2	TURN_START	0010
S3	WAIT_SELECTION	0011
S4	REVEAL	0100
S5	AUTO_PICK	0101
S6	EVALUATE_PAIR	0110

S7	MATCH	0111
S8	UNMATCH	1000
S9	HIDE_PAIR	1001
S10	GAME_OVER	1010

Banderas y señales de la máquina de estado:

- `clr_timers`: Limpia/reesetea los contadores de tiempo.
- `clr_score`: Reinicia el score de los jugadores.
- `init_board`: Inicializa el tablero de cartas.
- `random_enable`: Habilita el generador aleatorio para barajar.
- `board_write`: Escribe la barajada en el tablero.
- `shuffle_done`: Indica que las cartas están barajadas.
- `selCount[1:0]`: Contador del número de cartas seleccionadas en el turno (0, 1 o 2).
- `start_timer`: Inicia el temporizador de 15 s para el turno.
- `show_7seg`: Habilita la visualización del tiempo, scores en los displays de 7 segmentos.
- `scan_buttons`: Escanea los botones de entrada para detectar la jugada.
- `t15s_expired`: Indica que el tiempo de turno se agotó (15 s).
- `btn_valid`: Señala que la jugada ingresada por el jugador es válida.
- `random_pick`: Señala que se debe seleccionar una carta automáticamente (cuando expira el tiempo).
- `flip_sel_card`: Ordena voltear la carta seleccionada.
- `store_sel_card`: Guarda la carta seleccionada para evaluación posterior.
- `compare_enable`: Habilita la comparación de las dos cartas seleccionadas.
- `match`: Resultado de la comparación 1 si hay par, 0 si no.
- `lock_pair(pair1, pair2)`: Bloquea un par encontrado para que no pueda seleccionarse de nuevo.
- `inc_score[player]`: Incrementa el puntaje del jugador actual.
- `short_delay`: Señal para esperar un lapso pequeño de tiempo antes de ocultar cartas.
- `delay_done`: Fin del tiempo de espera del tiempo pequeño.
- `hide_cards`: Ordena voltear las cartas nuevamente (si no hubo match).
- `player = ~player`: Cambia el turno de jugador.
- `all_pairs`: Indica si ya se encontraron todas las parejas de cartas.
- `display_winner`: Señal para mostrar quién ganó.
- `reset`: Reinicio del sistema.

Propuesta 2

En la Propuesta 2, en este caso se organiza para manejar los turnos y la selección de cartas de manera más explícita. A diferencia de la primera propuesta, aquí se dividen las etapas de escoger la primera y la segunda carta. Con esto, permite verificar los pares, actualizar los puntajes, ocultar las cartas que no hacen match y cambiar el turno entre jugadores. Además, se agrega la señal `pairs_found` para llevar el conteo de los pares hasta llegar a 8 y se reasignan algunas señales a otros estados.

Controlador principal

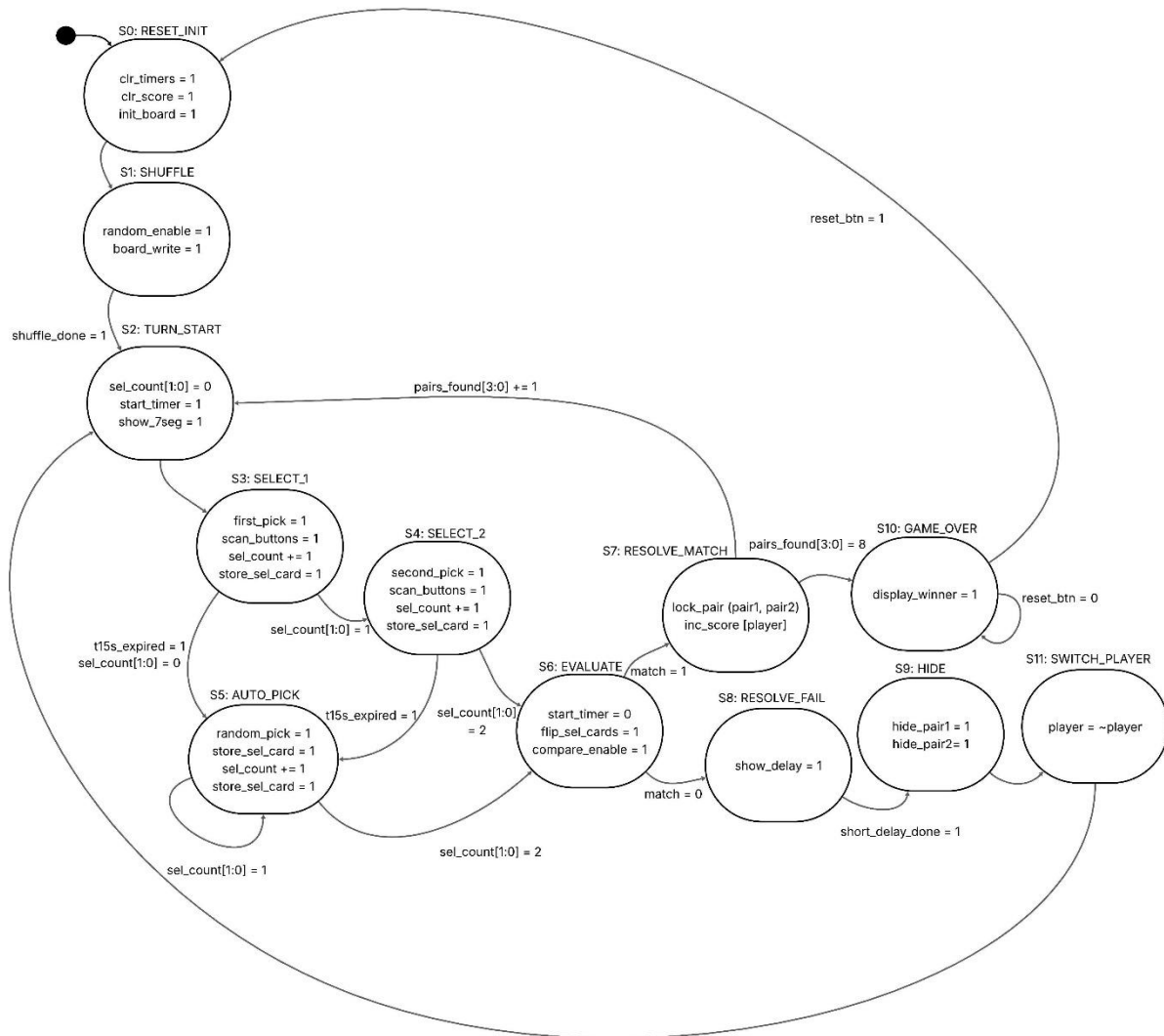


Figura 7. Máquina de Estado Finito para la propuesta 2.

A continuación, se muestra una tabla con la codificación de cada uno de los estados:

Estado	Nombre	Binario (4bits)
S0	RESET_INIT	0000
S1	SHUFFLE	0001
S2	TURN_START	0010
S3	SELECT_1	0011
S4	SELECT_2	0100
S5	AUTO_PICK	0101
S6	EVALUATE	0110
S7	RESOLVE_MATCH	0111
S8	RESOLVE_FAIL	1000
S9	HIDE	1001
S10	GAME_OVER	1010
S11	SWITCH_PLAYER	1011

Banderas y señales de la máquina de estado:

- `clr_timers`: Limpia/reesetea los contadores de tiempo.
- `clr_score`: Reinicia el score de los jugadores.
- `init_board`: Inicializa el tablero de cartas.
- `random_enable`: Habilita el generador aleatorio para barajar.
- `board_write`: Escribe la barajada en pantalla.
- `shuffle_done`: Indica que las cartas están barajadas.
- `start_timer`: inicia el temporizador de turno (15 s).
- `t15s_expired`: Indica que el tiempo de turno se agotó (15 s).
- `show_7seg`: habilita actualización de displays de 7 segmentos.
- `scan_buttons`: habilita lectura de botones.
- `first_pick`: la selección actual es la primera carta.
- `second_pick`: la selección actual es la segunda carta.
- `sel_count[1:0]`: número de cartas seleccionadas (0/1/2).
- `btn_valid`: Señala que la jugada ingresada por el jugador es válida.
- `random_pick`: Señala que se debe seleccionar una carta automáticamente (cuando se acaba el tiempo).
- `store_sel_card`: Guarda la carta seleccionada para evaluación posterior.
- `compare_enable`: Habilita la comparación de las dos cartas seleccionadas.
- `match`: Resultado de la comparación 1 si hay par, 0 si no.
- `lock_pair(pair1, pair2)`: Bloquea un par encontrado para que no pueda seleccionarse de nuevo.
- `flip_sel_cards`: asegura que ambas cartas estén visibles.
- `inc_score[player]`: incrementa puntaje del jugador actual.

- short_delay: Señal para esperar un lapso pequeño de tiempo antes de ocultar cartas.
- hide_pair1: oculta la primera carta (si no hubo match).
- hide_pair2: oculta la segunda carta (si no hubo match).
- short_delay_done: fin del tiempo de espera breve.
- player = ~player: alterna el jugador en turno.
- pairs_found[3:0]: contador de pares encontrados (0–8)
- display_winner: muestra el ganador (fin de juego).
- reset_btn: botón físico de reinicio.
- reset: reset del sistema.

Al comparar las dos propuestas de máquina de estados, notamos que la Propuesta 1 es más sencilla y directa. En lugar de separar cada paso en varios estados, agrupa algunas acciones y se apoya en banderas como sel_count para manejar la lógica de la selección de cartas. Esto hace que el diseño ocupe menos recursos en la FPGA y sea más fácil de implementar a nivel de hardware, manteniendo todo lo necesario.

En contraste, la Propuesta 2 desglosa de forma más detallada los turnos y las selecciones, creando estados específicos para la primera y segunda carta. Aunque esto facilita entender el flujo del juego desde un punto de vista lógico, también genera cierta redundancia y exige más señales de control. En la práctica, esto implica un mayor uso de recursos y un diseño un poco más pesado de sintetizar.

Tomando en cuenta lo anterior, se considera que la Propuesta 1 es la mejor opción. Es más eficiente en términos de hardware y, al mismo tiempo, logra mantener la claridad necesaria para que el sistema funcione correctamente.